Quiz 4

CS 320: Concepts of Programming Languages

November 4, 2026

Name: Nathan Mull

BUID: 12345678

- ▷ You will have approximately 35 minutes to complete this quiz. Make sure to read every question; some are easier than others.
- ▷ Make it very clear what your final solution for each problem is (e.g., by surrounding it in a box). We reserve the right to mark off points if we cannot tell what your final solution is.
- ▷ You must show your work on all problems unless otherwise specified. A solution without work will be considered incorrect (and will be investigated for potential academic misconduct).

1 OCaml Programming

Recall the ADT for λ-expressions:

```
type expr =
  | Lam of string * expr
  | App of expr * expr
  | Var of string
```

and the call-by-name (CBN) small-step semantics for the λ-calculus:

$$\frac{e_1 \longrightarrow e_2}{e_1 e \longrightarrow e_2 e}\ \textsc{appLeft} \qquad\qquad \frac{}{(\lambda x.e_1)e_2 \longrightarrow [e_2/x]e_1}\ \textsc{beta}$$

Implement the function

```
val cbn_step : expr -> expr option
```

so that `cbn_step e` is `Some e'` if $e \longrightarrow e'$ according to the above CBN semantics, and `None` otherwise. You may assume access to `subst : expr -> string -> expr -> expr` where `subst e2 x e1` is $[e_2/x]e_1$.

```
let cbn_step e =
  match e with
  | Var _ -> None
  | Lam (_, _) -> None
  | App (Lam (x, e1), e2) -> Some (subst e2 x e1)
  | App (e1, e) ->
      match cbn_step e1 with
      | Some e2 -> Some (App (e2, e))
      | None -> None
```

2 Capture-Avoiding Substitution

Recall the definition of capture-avoiding substitution in the λ-calculus.

$$[e/x](\lambda y.e_1) = \begin{cases} \lambda y.e_1 & x = y \\ \lambda y.[e/x]e_1 & x \neq y, y \notin \mathsf{FV}(e) \\ \lambda z.[e/x][z/y]e_1 & x \neq y, y \in \mathsf{FV}(e), z \text{ is fresh} \end{cases}$$

$$[e/x](e_1 e_2) = ([e/x]e_1)([e/x]e_2)$$

$$[e/x]y = \begin{cases} e & x = y \\ y & x \neq y \end{cases}$$

Perform the following substitutions. You're free to choose any fresh variable name as necessary. You must show your work. You don't need to show every single step, but you must show some steps so it's clear you know how to use the above definition.

A. $[yz/w]((\lambda y.w)(\lambda z.w))$

B. $[\lambda x.y/z](\lambda y.\lambda x.zy)$

A. $[yz/w]((\lambda y.w)(\lambda z.w))$

$= ([yz/w](\lambda y.w))([yz/w](\lambda z.w))$

$= (\lambda q.[yz/w][q/y]w)(\lambda r.[yz/w][r/z]w)$

$= (\lambda q.[yz/w]w)(\lambda r.[yz/w]w)$

$= (\lambda q.yz)(\lambda r.yz)$

B. $[\lambda x.y/z](\lambda y.\lambda x.zy)$

$= \lambda w.[\lambda x.y/z][w/y](\lambda x.zy)$

$= \lambda w.[\lambda x.y/z](\lambda x.[w/y](zy))$

$= \lambda w.[\lambda x.y/z](\lambda x.([w/y]z)([w/y]y))$

$= \lambda w.[\lambda x.y/z](\lambda x.zw)$

$= \lambda w.\lambda x.[\lambda x.y/z](zw) = \lambda w.\lambda x.(\lambda x.y)w$

3 CBV vs. CBN

Recall the call-by-value (CBV) big-step semantic rules for the λ-calculus:

$$\frac{}{\lambda x.e \Downarrow \lambda x.e}\ \text{FUN} \qquad \frac{e_1 \Downarrow \lambda x.e \qquad e_2 \Downarrow v_2 \qquad [v_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}\ \text{APPCBV}$$

and the call-by-name (CBN) big-step semantic rules for the λ-calculus:

$$\frac{}{\lambda x.e \Downarrow \lambda x.e}\ \text{FUN} \qquad \frac{e_1 \Downarrow \lambda x.e \qquad [e_2/x]e \Downarrow v}{e_1 e_2 \Downarrow v}\ \text{APPCBN}$$

Determine a value v such that $(\lambda x.\lambda y.y)((\lambda z.z)(\lambda q.q)) \Downarrow v$ under both CBV and CBN. Also provide derivations under both semantics. Please label which derivation is which.

CBV

$$\dfrac{\dfrac{}{\lambda x.\lambda y.y \Downarrow \lambda x.\lambda y.y}\,\text{fun} \qquad \dfrac{\dfrac{}{\lambda z.z \Downarrow \lambda z.z}\,\text{fun} \qquad \dfrac{}{\lambda q.q \Downarrow \lambda q.q}\,\text{fun} \qquad \dfrac{}{\lambda q.q \Downarrow \lambda q.q}\,\text{fun}}{(\lambda z.z)(\lambda q.q) \Downarrow \lambda q.q}\,\text{app} \qquad \dfrac{}{\lambda y.y \Downarrow \lambda y.y}\,\text{fun}}{(\lambda x.\lambda y.y)((\lambda z.z)(\lambda q.q)) \Downarrow \lambda y.y}\,\text{app}$$

CBN

$$\dfrac{\dfrac{}{\lambda x.\lambda y.y \Downarrow \lambda x.\lambda y.y}\,\text{fun} \qquad \dfrac{}{\lambda y.y \Downarrow \lambda y.y}\,\text{fun}}{(\lambda x.\lambda y.y)((\lambda z.z)(\lambda q.q)) \Downarrow \lambda y.y}\,\text{app}$$